

RenderSurfaceLighting()

```
for all lights do  
    Clear Depth Texture  
    for all surfaces do  
        Generate MVP  
        WriteDepthBuffer()  
    end for  
end for
```

WriteDepthBuffer()

```
Set shader program patch_tess_pass  
Assign SSBO's and uniforms, light_pass = 1  
for all surface control points (concurrently) do  
    Dispatch compute shader patch_tess_pass  
    Get a triangulated version of our patch control points as an EBO  
end for  
Set shader program patch_shadow  
Render surface as triangulated control points to depth buffer  
if We want to visualize triangulated tiles then  
    Set shader program visualize_tiles  
    Render triangulated tiles to primary camera  
end if
```

patch_tess_pass light pass

```
Interpret raw control point vertex data  
Calculate 2nd derivative in the U direction  
Calculate 2nd derivative in the V direction  
for all tiles do  
    Get indices to construct two triangles representing the tile quad  
    Emplace those six indices into the EBO with control points as vertices  
    We are triangulating our control points into a mesh here to use for  
    shadow mapping  
end for
```

patch_shadow

```
for all vertices do
```

```

    Project from light source
    Write to depth buffer
end for

```

```

RenderSurfaces()
for all surfaces do
    UpdatePatchTessLevels()
    RenderSurface()
end for

```

```

UpdatePatchTessLevels()
Set shader program patch_tess_pass
Assign SSBO's and uniforms, light_pass = 0
for all surface control points (concurrently) do
    Dispatch compute shader patch_tess_pass
end for

```

```

patch_tess_pass tess pass
Interpret raw control point vertex data
Calculate 2nd derivative in the U direction
Calculate 2nd derivative in the V direction
for all patches do
    for all 9 tiles composing the patch, simultaneously do
        Compute the average center box of tile's bilinear breakpoints
    end for
    Get corner of avg center box closest to the camera to get minimal
    screen-coordinate-aligned rectangle that covers slefe box
    Apply minimal tessellation factor that achieves parametric-accuracy
    if patch is entirely out of view then
        Set tessellation factor to 0.0
    end if
end for

```

```

RenderSurface()
Set shader program render_pass

```

Assign SSBO's and uniforms

Draw Elements (render_pass.vs, .tcs, .tes, .fs)